



School of Computer Science

COMP47470: Big Data Programming

Neo4j NOSQL DBMS and Cypher Query Language Lab Practical 1

Module Coordinator	Ravi Reddy Manumachu
Email	ravi.manumachu@ucd.ie

Introduction: Installation, Service Checks	2
Neo4j Movies	2
Cypher Terminology for Property Graph	3
Loading Movies Property Graph	3
Queries to Understand Property Graph Structure	5
Basic Queries	8
Advanced Queries	11
Queries to Solve	13
Path Finding in Neo4j	13
Stations and Stops Property Graph	14
Matching Fixed-Length Paths	15
Matching Shortest Paths	16
Queries to Solve	18

Introduction: Installation, Service Checks

Install Neo4j using the Installation and Quick Start guide in '**About the Tools**' in Brightspace. Before attempting the practical, set the password for the default user **neo4j**.

The guide's instructions are primarily intended for Ubuntu Linux. However, you are not required to use Unix/Linux platforms for the Neo4j practicals; Windows and Mac machines are also acceptable.

Ensure that you can start the Neo4j service and have access to tools like `neo4j_admin` and `neo4j_shell` to execute Neo4j commands.

Check the status of Neo4j service:

\$ systemctl status neo4j

```
manumachu@system76-pc: ~/comp47470/SoftwarePackages$ systemctl status neo4j
● neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-21 18:13:43 IST; 9min ago
     Main PID: 2219 (java)
       Tasks: 107 (limit: 38184)
      Memory: 1.0G (peak: 1.0G)
         CPU: 22.654s
      CGroup: /system.slice/neo4j.service
              └─2219 /usr/bin/java -Xmx128m -XX:+IgnoreUnrecognizedVMOptions --sun-misc-unsafe-memory-access=allow -classpath "/usr/share/neo4j/lib/*:/usr/share/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/*" -XX:+UseG1GC
                └─2685 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp "/var/lib/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/*" -XX:+UseG1GC

Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.597+0000 INFO Anonymous Usage Data is being sent to Neo4j, see https://neo4j.com/docs/
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.746+0000 INFO Bolt enabled on localhost:7687.
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.755+0000 INFO Fleet discovery broadcasts have been enabled for 3 networks: 172.17.0.0/16, 10.0.0.0/8, 192.168.0.0/24
Sep 21 18:13:46 system76-pc neo4j[2685]: 2026-09-21 17:13:46.981+0000 INFO This server has built-in support for online monitoring through Prometheus
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.141+0000 INFO HTTP enabled on localhost:7474.
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.142+0000 INFO Remote interface available at http://localhost:7474/
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO id: 048B00E6530EFB55746B7E505400EA773DB6ABF0CA6FC8495D883526A6D9
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO name: system
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO creationDate: 2026-07-22T08:53:49.604Z
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO Started.
```

You can start and stop the neo4j service as follows:

```
$ sudo systemctl start neo4j
```

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ systemctl status neo4j
○ neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2026-09-21 18:25:42 IST; 1min 1s ago
   Duration: 11min 48.349s
   Process: 2219 ExecStart=/usr/share/neo4j/bin/neo4j console (code=exited, status=0/SUCCESS)
   Main PID: 2219 (code=exited, status=0/SUCCESS)
   CPU: 27.938s

Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO name: system
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO creationDate: 2026-07-22T08:53:49.604Z
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO Started.
Sep 21 18:25:31 system76-pc systemd[1]: Stopping neo4j.service - Neo4j Graph Database...
Sep 21 18:25:31 system76-pc neo4j[2685]: 2026-09-21 17:25:31.985+0000 INFO Neo4j Server shutdown initiated by request
Sep 21 18:25:31 system76-pc neo4j[2685]: 2026-09-21 17:25:31.988+0000 INFO Stopping...
Sep 21 18:25:42 system76-pc neo4j[2685]: 2026-09-21 17:25:42.209+0000 INFO Stopped.
Sep 21 18:25:42 system76-pc systemd[1]: neo4j.service: Deactivated successfully.
Sep 21 18:25:42 system76-pc systemd[1]: Stopped neo4j.service - Neo4j Graph Database.
Sep 21 18:25:42 system76-pc systemd[1]: neo4j.service: Consumed 27.938s CPU time, 1.0G memory peak, 0B memory swap peak.
```

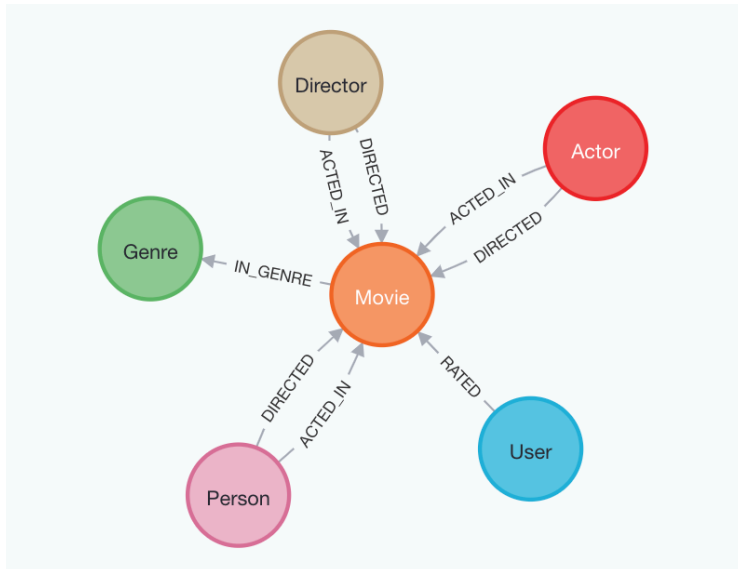
```
$ sudo systemctl stop neo4j
```

Let us start working with some Neo4j databases and run some graph queries.

You are provided a zip file containing the Neo4j database dumps. Untar the zip file.

Neo4j Movies

We will load the **movies** property graph. Note we will use the terms property graph and database interchangeably. The property graph and pictorial representation of a query result are shown below:



Cypher Terminology for Property Graph

The above movie property graph has **six** primary node types or labels. The labels are Person, Movie, User, Genre, Actor and Director.

In this practical, we will work with two node labels and some relationships between them.

Each node type/label has specific properties:

Person represents individuals with properties **name** and **born**.

Movie represents films with properties **title**, **released**, and **tagline**.

Six relationship types connect Person and Movie nodes. These are: ACTED_IN, DIRECTED, PRODUCED, WROTE, FOLLOWS, and REVIEWED.

Each relationship type has properties. **ACTED_IN** has roles property. **REVIEWED** has summary and rating properties. The other relationship types have no properties.

Loading Movies Property Graph

But first, check the contents of the neo4j database. You must make sure it is empty before you attempt to run queries on the property graphs in this tutorial/practical.

\$ cypher-shell -u neo4j -p neo4j123 "SHOW DATABASES"

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ cypher-shell -u neo4j -p neo4j123 "SHOW DATABASES"
+-----+
| name | type | aliases | access | address | role | writer | requestedStatus | currentStatus | statusMessage |
+-----+
| "neo4j" | "standard" | [] | "read-write" | "localhost:7687" | "primary" | TRUE | "online" | "online" | "" |
| "system" | "system" | [] | "read-write" | "localhost:7687" | "primary" | TRUE | "online" | "online" | "" |
+-----+
2 rows
ready to start consuming query after 411 ms, results consumed after another 47 ms
```

The neo4j database is your property graph, while system is a metadata database that cannot run graph queries.

The neo4j database is the default database where your actual graph data (nodes, relationships, properties) is stored.

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ cypher-shell -u neo4j -p neo4j123
Connected to Neo4j using Bolt protocol version 6.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> :use neo4j
neo4j@neo4j> MATCH (n) RETURN n LIMIT 10;
+-----+
| n                                           |
+-----+
| (:Station {name: "Peckham Rye"})           |
| (:Station {name: "Denmark Hill"})          |
| (:Station {name: "London Victoria"})       |
| (:Station {name: "Clapham High Street"})   |
| (:Station {name: "Elephant & Castle"})     |
| (:Stop {departs: 11:55Z})                  |
| (:Stop {departs: 11:44Z})                  |
| (:Stop {departs: 11:40Z})                  |
| (:Stop {departs: 11:41Z})                  |
| (:Stop {departs: 11:37Z})                  |
+-----+
10 rows
ready to start consuming query after 18 ms, results consumed after another 2 ms
neo4j@neo4j>
```

It is not empty. You must delete the contents because the neo4j community edition allows to work with just one property graph.

Either execute the following query in the neo4j shell:

```
neo4j@neo4j> MATCH (n) DETACH DELETE n;
```

0 rows

ready to start consuming query after 186 ms, results consumed after another 0 ms

Deleted 14 nodes, deleted 15 relationships

Or execute the following outside the neo4j shell:

```
cypher-shell -u neo4j -p neo4j123 "MATCH (n) DETACH DELETE n;"
```

You need to remove the following specific directories for the neo4j database. The exact paths depend on your Neo4j version and installation, but they are typically:

Databases: /var/lib/neo4j/data/databases/neo4j

Transactions: /var/lib/neo4j/data/transactions/neo4j

```
$ sudo rm -rf /var/lib/neo4j/data/databases/neo4j
```

```
$ sudo rm -rf /var/lib/neo4j/data/transactions/neo4j
```

Load the database using the **neo4j-admin** tool from the movies directory. Before that, you must stop the neo4j service.

```
$ sudo systemctl stop neo4j
```

```
$ cp Neo4j_LabPractical1_DBs/movies-50.dump /tmp/neo4j.dump
```

```
$ sudo -u neo4j neo4j-admin database load neo4j --from-path=/tmp --overwrite-destination=true
```

Done: 83 files, 1.134MiB processed in 0.078 seconds.

Start the neo4j service and run the queries in **cypher-shell**.

```
$ sudo systemctl start neo4j
```

```
$ cypher-shell -u neo4j -p neo4j123
```

Queries to Understand Property Graph Structure

Let us understand the structure of the graph.

1. List all node labels.

CALL db.labels();

```
neo4j@neo4j> CALL db.labels();
+-----+
| label |
+-----+
| "Person" |
| "Movie" |
+-----+

2 rows
ready to start consuming query after 63 ms, results consumed after another 4 ms
```

2. List all relationship types.

CALL db.relationshipTypes();

```
neo4j@neo4j> CALL db.relationshipTypes();
+-----+
| relationshipType |
+-----+
| "ACTED_IN" |
| "DIRECTED" |
| "PRODUCED" |
| "WROTE" |
| "FOLLOWS" |
| "REVIEWED" |
+-----+

6 rows
ready to start consuming query after 49 ms, results consumed after another 2 ms
```

3. List all property keys to see what attributes are stored on nodes and relationships.

CALL db.propertyKeys();

```
neo4j@neo4j> CALL db.propertyKeys();
+-----+
| propertyKey |
+-----+
| "name" |
| "born" |
| "title" |
| "released" |
| "tagline" |
| "roles" |
| "summary" |
| "rating" |
+-----+

8 rows
ready to start consuming query after 43 ms, results consumed after another 2 ms
```

4. You can use the **db.schema.nodeTypeProperties()** procedure to retrieve the properties associated with each node label

CALL db.schema.nodeTypeProperties();

nodeType	nodeLabels	propertyName	propertyTypes	mandatory
"`Movie`"	["Movie"]	"title"	["STRING NOT NULL"]	TRUE
"`Movie`"	["Movie"]	"released"	["INTEGER NOT NULL"]	TRUE
"`Movie`"	["Movie"]	"tagline"	["STRING NOT NULL"]	FALSE
"`Person`"	["Person"]	"name"	["STRING NOT NULL"]	TRUE
"`Person`"	["Person"]	"born"	["INTEGER NOT NULL"]	FALSE

- To find the properties associated with relationship types in Neo4j, you can use the built-in procedure **db.schema.relTypeProperties()**, which is the direct counterpart to the node procedure you already used.

CALL db.schema.relTypeProperties();

relType	propertyName	propertyTypes	mandatory
"`ACTED_IN`"	"roles"	["LIST<STRING NOT NULL> NOT NULL"]	TRUE
"`DIRECTED`"	NULL	NULL	FALSE
"`PRODUCED`"	NULL	NULL	FALSE
"`WROTE`"	NULL	NULL	FALSE
"`FOLLOWS`"	NULL	NULL	FALSE
"`REVIEWED`"	"summary"	["STRING NOT NULL"]	TRUE
"`REVIEWED`"	"rating"	["INTEGER NOT NULL"]	TRUE

- Count nodes per label to see how many of each type of node exist.

MATCH (n)

RETURN labels(n) AS label, count(*) AS count

ORDER BY count DESC;

Let us understand the basic query above on finding nodes.

Nodes are referred to in Cypher using parentheses **(n)**. The **MATCH** clause is used to find a specific pattern in the graph, such as a specific node. The **RETURN** clause specifies which of the found graph patterns to return.

MATCH (n) Finds every node in the database. The (n) matches any node with no label or property filter.

RETURN labels(n) AS label For each node, extracts its label (e.g., Person, Movie) and renames the output column to label.

count(*) AS count Counts how many nodes share the same label. The count(*) aggregates rows that have identical label values — this is the implicit grouping.

ORDER BY count DESC Sorts the result so the label with the most nodes appears at the top.

```
neo4j@neo4j> MATCH (n)
RETURN labels(n) AS label, count(*) AS count
ORDER BY count DESC;
+-----+
| label      | count |
+-----+
| ["Person"] | 133   |
| ["Movie"]  | 38    |
+-----+

2 rows
ready to start consuming query after 345 ms, results consumed after another 19 ms
```

- Count relationships per type to find out how many of each kind of relationship exist, revealing which connections dominate the graph.

MATCH ()-[r]->()

RETURN type(r) AS relationshipType, count(*) AS count
ORDER BY count DESC;

Let us figure out the basic query above on finding relationships.

To discover how nodes are connected to one another, relationships must be added to queries. Queries can specify relationship types, properties, and direction, as well as the start and end nodes of the pattern.

Relationships are represented in Cypher with arrows (e.g. ->) indicating the direction of a relationship.

MATCH ()-[r]->() Finds every relationship in the database. The empty parentheses () mean "any node, with no label or property filter" — so this matches any node on either end of the relationship.

RETURN type(r) AS relationshipType Extracts the relationship's type (e.g., ROUTE, OPERATES, ACTED_IN) and renames the output column to relationshipType.

count(*) AS count Counts how many relationships share the same type. The implicit grouping is on the relationshipType value.

ORDER BY count DESC Sorts the result so the most frequent relationship type appears at the top.

```
neo4j@neo4j> MATCH ()-[r]->()
RETURN type(r) AS relationshipType, count(*) AS count
ORDER BY count DESC;
+-----+
| relationshipType | count |
+-----+
| "ACTED_IN"       | 172   |
| "DIRECTED"       | 44    |
| "PRODUCED"       | 15    |
| "WROTE"          | 10    |
| "REVIEWED"       | 9     |
| "FOLLOWS"        | 3     |
+-----+

6 rows
ready to start consuming query after 166 ms, results consumed after another 7 ms
```

- Show one concrete example of a relationship and, crucially, which labels it connects.

MATCH (a)-[r]->(b)

RETURN type(r) AS relationshipType,
labels(a) AS fromLabels,
labels(b) AS toLabels,

a, r, b

LIMIT 1;

```
neo4j@neo4j> MATCH (a)-[r]->(b)
                RETURN type(r) AS relationshipType,
                        labels(a) AS fromLabels,
                        labels(b) AS toLabels,
                        a, r, b
                LIMIT 1;
+-----+
| relationshipType | fromLabels | toLabels | a | r | b |
+-----+
| "ACTED_IN"      | ["Person"] | ["Movie"] | (:Person {name: "Keanu Reeves", born: 1964}) | [:ACTED_IN {roles: ["Neo"]}] | (:Movie {title: "The Matrix", tagline: "Welcome to the Real World", released: 1999}) |
+-----+
1 row
ready to start consuming query after 75 ms, results consumed after another 2 ms
```

Basic Queries

1. Find a specific actor. The following query finds the node with the Person label and the name property "Tom Hanks".

MATCH (tom:Person {name: "Tom Hanks"})

RETURN tom

Let us understand the basic query above.

MATCH Tells Neo4j to search the graph for a pattern that matches what follows.

(tom:Person {name: "Tom Hanks"}) Looks for a node that has the label Person and has a name property equal to "Tom Hanks". The variable tom is just a name you assign to the matched node so you can refer to it later.

RETURN tom Outputs the matched node. In cypher-shell, it prints the node's internal ID and property list.

```
neo4j@neo4j> MATCH (tom:Person {name: "Tom Hanks"})
                RETURN tom;
+-----+
| tom |
+-----+
| (:Person {name: "Tom Hanks", born: 1956}) |
+-----+
1 row
ready to start consuming query after 233 ms, results consumed after another 17 ms
```

2. Find a specific movie by locating a Movie node by its title.

MATCH (cloudAtlas:Movie {title: "Cloud Atlas"})

RETURN cloudAtlas

```
neo4j@neo4j> MATCH (cloudAtlas:Movie {title: "Cloud Atlas"})
                RETURN cloudAtlas;
+-----+
| cloudAtlas |
+-----+
| (:Movie {title: "Cloud Atlas", tagline: "Everything is connected", released: 2012}) |
+-----+
1 row
ready to start consuming query after 59 ms, results consumed after another 2 ms
```

3. Find movies released in a specific decade.

MATCH (nineties:Movie)
WHERE nineties.released > 1990 AND nineties.released < 2000
RETURN nineties.title;

Let us understand the basic query above.

MATCH (nineties:Movie) Finds every node with the label Movie. The variable nineties is just a name for the matched node - it doesn't filter anything by itself.

WHERE nineties.released > 1990 AND nineties.released < 2000 Keeps only the movies whose released property is greater than 1990 and less than 2000. This is the actual filter.

RETURN nineties.title Outputs the title property of each matching movie.

```
neo4j@neo4j> MATCH (nineties:Movie)
               WHERE nineties.released > 1990 AND nineties.released < 2000
               RETURN nineties.title;
+-----+
| nineties.title |
+-----+
| "A Few Good Men" |
| "Unforgiven" |
| "Hoffa" |
| "A League of Their Own" |
| "Sleepless in Seattle" |
| "Johnny Mnemonic" |
| "Apollo 13" |
| "That Thing You Do" |
| "The Birdcage" |
| "Twister" |
| "The Devil's Advocate" |
| "As Good as It Gets" |
| "What Dreams May Come" |
| "You've Got Mail" |
| "When Harry Met Sally" |
| "The Matrix" |
| "Snow Falling on Cedars" |
+-----+
```

4. List all movies that Tom Hanks acted in. This query traverses a relationship, following the **ACTED_IN** connection from a person to movies.

MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->(tomHanksMovies)
RETURN tomHanksMovies.title;

Let us understand the basic query above.

(tom:Person {name: "Tom Hanks"}) Finds the node for Tom Hanks.

-[:ACTED_IN]-> Follows a relationship outward from Tom Hanks, keeping only relationships of type ACTED_IN. The arrow -> means the relationship points from Tom Hanks to the movie.

(tomHanksMovies) Binds each movie reached this way to the variable tomHanksMovies. Note there's no label here - it matches any node connected via ACTED_IN.

RETURN tomHanksMovies.title Outputs the title property of each movie.

```
neo4j@neo4j> MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->(tomHanksMovies)
RETURN tomHanksMovies.title;
+-----+
| tomHanksMovies.title |
+-----+
| "Apollo 13"          |
| "You've Got Mail"    |
| "A League of Their Own" |
| "Joe Versus the Volcano" |
| "That Thing You Do"   |
| "The Da Vinci Code"   |
| "Cloud Atlas"         |
| "Cast Away"           |
| "The Green Mile"       |
| "Sleepless in Seattle" |
| "The Polar Express"   |
| "Charlie Wilson's War" |
+-----+
12 rows
ready to start consuming query after 111 ms, results consumed after another 19 ms
```

- Find who directed a specific movie. The query looks for relationships pointing to the movie, using a backwards arrow.

```
MATCH (m:Movie {title: "The Matrix"})<-[d:DIRECTED]-(p:Person)
RETURN p.name AS director;
```

Let us understand this different directional query.

(m:Movie {title: "The Matrix"}) Finds the node with the Movie label whose title property is "The Matrix". Binds it to the variable m.

<-[d:DIRECTED]- Follows a DIRECTED relationship inward - that is, toward the movie. The arrow points from the director to the movie, so to traverse from the movie back to the director, you reverse the arrow. The variable d is bound to the relationship itself.

(p:Person) Binds the person at the other end to the variable p. The label :Person ensures we only match actual people, not some other node type.

RETURN p.name AS director Outputs the person's name property, renaming the column to director for readability.

```
neo4j@neo4j> MATCH (m:Movie {title: "The Matrix"})<-[d:DIRECTED]-(p:Person)
RETURN p.name AS director;
+-----+
| director |
+-----+
| "Lana Wachowski" |
| "Lilly Wachowski" |
+-----+
2 rows
ready to start consuming query after 126 ms, results consumed after another 3 ms
```

- Find co-actors of a specific person. This query finds a pattern with multiple relationships: actors who acted in the same movie as **Tom Hanks**. It starts at Tom Hanks, finds his movies, and then finds other Person nodes connected to those same movies via **ACTED_IN**.

```
MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(costar:Person)
RETURN DISTINCT costar.name;
```

```
neo4j@neo4j> MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(costar:Person)
RETURN DISTINCT costar.name;
+-----+
| costar.name |
+-----+
| "Ed Harris" |
| "Gary Sinise" |
| "Kevin Bacon" |
| "Bill Paxton" |
| "Parker Posey" |
| "Greg Kinnear" |
| "Meg Ryan" |
| "Steve Zahn" |
| "Dave Chappelle" |
| "Madonna" |
| "Rosie O'Donnell" |
| "Geena Davis" |
| "Lori Petty" |
| "Nathan Lane" |
| "Liv Tyler" |
+-----+
```

- Find how actors are connected through movies. This query looks for a two-hop pattern: an actor, the movie they acted in, and the director of that movie. This is a classic path traversal that combines two different relationship types.

```
MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(director:Person)
RETURN actor.name, movie.title, director.name
LIMIT 5;
```

```
neo4j@neo4j> MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(director:Person)
RETURN actor.name, movie.title, director.name
LIMIT 5;
+-----+-----+-----+
| actor.name | movie.title | director.name |
+-----+-----+-----+
| "Emil Eifrem" | "The Matrix" | "Lana Wachowski" |
| "Hugo Weaving" | "The Matrix" | "Lana Wachowski" |
| "Laurence Fishburne" | "The Matrix" | "Lana Wachowski" |
| "Carrie-Anne Moss" | "The Matrix" | "Lana Wachowski" |
| "Keanu Reeves" | "The Matrix" | "Lana Wachowski" |
+-----+-----+-----+

5 rows
ready to start consuming query after 109 ms, results consumed after another 2 ms
```

Advanced Queries

- Find movies that have both a director and at least one actor. This query uses a two-hop pattern with a **DISTINCT** to avoid duplicates.

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
RETURN DISTINCT m.title AS movie, d.name AS director
ORDER BY movie
LIMIT 5;
```

```
neo4j@neo4j> MATCH (d:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
RETURN DISTINCT m.title AS movie, d.name AS director
ORDER BY movie
LIMIT 5;
+-----+-----+
| movie | director |
+-----+-----+
| "A Few Good Men" | "Rob Reiner" |
| "A League of Their Own" | "Penny Marshall" |
| "Apollo 13" | "Ron Howard" |
| "As Good as It Gets" | "James L. Brooks" |
| "Bicentennial Man" | "Chris Columbus" |
+-----+-----+

5 rows
ready to start consuming query after 124 ms, results consumed after another 2 ms
```

- Find actors who have worked with a given director more than once. This query traverses two relationships and groups by the actor to find repeat collaborations. It uses a two-hop traversal, WITH for intermediate aggregation, WHERE filtering on an aggregate, and collect() to return a list.

```
MATCH (d:Person {name: "Clint Eastwood"})-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
WITH a, count(m) AS collaborations, collect(m.title) AS movies
WHERE collaborations >= 1
RETURN a.name AS actor, collaborations, movies
ORDER BY collaborations DESC;
```

```
neo4j@neo4j> MATCH (d:Person {name: "Clint Eastwood"})-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Person)
WITH a, count(m) AS collaborations, collect(m.title) AS movies
WHERE collaborations >= 1
RETURN a.name AS actor, collaborations, movies
ORDER BY collaborations DESC;

+-----+-----+-----+
| actor          | collaborations | movies          |
+-----+-----+-----+
| "Clint Eastwood" | 1              | ["Unforgiven"] |
| "Gene Hackman"   | 1              | ["Unforgiven"] |
| "Richard Harris" | 1              | ["Unforgiven"] |
+-----+-----+-----+

3 rows
ready to start consuming query after 115 ms, results consumed after another 3 ms
```

- Find pairs of actors who have acted in the most movies together. This query is a self-join pattern query. It uses self-join with a WHERE a1.name < a2.name trick avoids duplicate pairs, aggregation across a shared node, and filtering.

```
MATCH (a1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(a2:Person)
WHERE a1.name < a2.name
WITH a1, a2, count(m) AS sharedMovies, collect(m.title) AS movies
WHERE sharedMovies >= 3
RETURN a1.name AS actor1, a2.name AS actor2, sharedMovies, movies
ORDER BY sharedMovies DESC
LIMIT 10;
```

```
neo4j@neo4j> MATCH (a1:Person)-[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(a2:Person)
WHERE a1.name < a2.name
WITH a1, a2, count(m) AS sharedMovies, collect(m.title) AS movies
WHERE sharedMovies >= 3
RETURN a1.name AS actor1, a2.name AS actor2, sharedMovies, movies
ORDER BY sharedMovies DESC
LIMIT 10;

+-----+-----+-----+-----+
| actor1          | actor2          | sharedMovies | movies          |
+-----+-----+-----+-----+
| "Carrie-Anne Moss" | "Hugo Weaving" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Hugo Weaving"    | "Laurence Fishburne" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Carrie-Anne Moss" | "Laurence Fishburne" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Keanu Reeves"    | "Laurence Fishburne" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Hugo Weaving"    | "Keanu Reeves" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Carrie-Anne Moss" | "Keanu Reeves" | 3            | ["The Matrix", "The Matrix Reloaded", "The Matrix Revolutions"] |
| "Meg Ryan"        | "Tom Hanks"      | 3            | ["You've Got Mail", "Sleepless in Seattle", "Joe Versus the Volcano"] |
+-----+-----+-----+-----+
```

- Find actors who have never directed a movie. The query returns the names of people who have ACTED_IN at least one movie but have never DIRECTED any movie at all.

```
MATCH (a:Person)-[:ACTED_IN]->(:Movie)
WITH DISTINCT a
WHERE NOT EXISTS {
  MATCH (a)-[:DIRECTED]->(:Movie)
}
RETURN a.name AS actorOnly
```

ORDER BY actorOnly
LIMIT 5;

```
neo4j@neo4j> MATCH (a:Person)-[:ACTED_IN]->(:Movie)
               WITH DISTINCT a
               WHERE NOT EXISTS {
                 MATCH (a)-[:DIRECTED]->(:Movie)
               }
               RETURN a.name AS actorOnly
               ORDER BY actorOnly
               LIMIT 5;

+-----+
| actorOnly |
+-----+
| "Aaron Sorkin" |
| "Al Pacino"    |
| "Annabella Sciorra" |
| "Anthony Edwards" |
| "Audrey Tautou"  |
+-----+

5 rows
ready to start consuming query after 132 ms, results consumed after another 7 ms
```

Queries to Solve

1. Write a query that finds movies with exactly two actors?

Hint: Use count(DISTINCT ...) to count unique neighbors, plus filtering after a WITH.

2. Write a query to find actors who acted in more than one movie in the same year?

Hint: Group by both the actor and the release year, then filter on the count.

3. Write a query to find movies that have no actors?

Hint: Use NOT EXISTS { ... } with a pattern inside the subquery.

Path Finding in Neo4j

Let us empty the Neo4j database before creating the next property graph.

Either execute the following query in the neo4j shell:

```
neo4j@neo4j> MATCH (n) DETACH DELETE n;
0 rows
ready to start consuming query after 186 ms, results consumed after another 0 ms
Deleted 14 nodes, deleted 15 relationships
```

Or execute the following outside the neo4j shell:

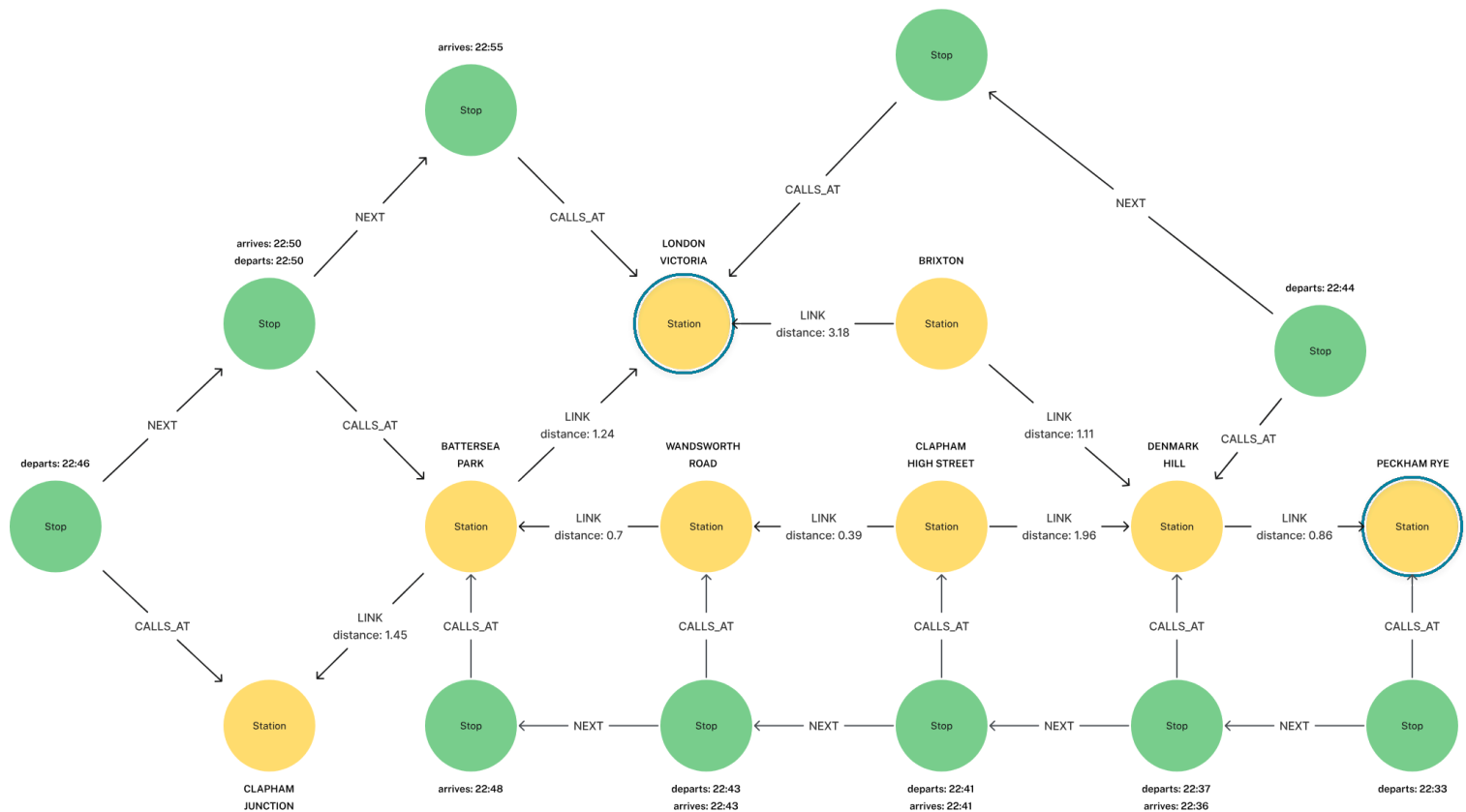
```
cypher-shell -u neo4j -p neo4j123 "MATCH (n) DETACH DELETE n;"
```

You need to remove the following specific directories for the neo4j database. The exact paths depend on your Neo4j version and installation, but they are typically:

Databases: /var/lib/neo4j/data/databases/neo4j
Transactions: /var/lib/neo4j/data/transactions/neo4j

```
$ sudo rm -rf /var/lib/neo4j/data/databases/neo4j  
$ sudo rm -rf /var/lib/neo4j/data/transactions/neo4j
```

Stations and Stops Property Graph



To create the above graph, run the following query against the empty Neo4j database in cypher-shell:

```
CREATE (n1:Station {name: 'Denmark Hill'}),
(n5:Station {name: 'Battersea Park'}),
(n6:Station {name: 'Wandsworth Road'}),
(n15:Station {name: 'Clapham High Street'}),
(n16:Station {name: 'Peckham Rye'}),
(n17:Station {name: 'Brixton'}),
(n14:Station {name: 'London Victoria'}),
(n18:Station {name: 'Clapham Junction'}),
(p10:Stop {departs: time('22:37'), arrives: time('22:36')}),
(p0:Stop {departs: time('22:41'), arrives: time('22:41')}),
(p2:Stop {departs: time('22:43'), arrives: time('22:43')}),
(p17:Stop {arrives: time('22:50'), departs: time('22:50')}),
(p18:Stop {arrives: time('22:46'), departs: time('22:46')}),
(p19:Stop {departs: time('22:33'), arrives: time('22:31')}),
(p21:Stop {arrives: time('22:55')}),
(p20:Stop {departs: time('22:44'), arrives: time('22:43')}),
(p22:Stop {arrives: time('22:55')}),
(p23:Stop {arrives: time('22:48')}),
(n15)-[:LINK {distance: 1.96}]->(n1)-[:LINK {distance: 0.86}]->(n16),
(n15)-[:LINK {distance: 0.39}]->(n6)-[:LINK {distance: 0.7}]->(n5)-[:LINK {distance: 1.24}]->(n14),
(n5)-[:LINK {distance: 1.45}]->(n18),
(n14)-[:LINK {distance: 3.18}]->(n17)-[:LINK {distance: 1.11}]->(n1),
(p2)-[:CALLS_AT]->(n6), (p17)-[:CALLS_AT]->(n5), (p19)-[:CALLS_AT]->(n16),
(p22)-[:CALLS_AT]->(n14), (p18)-[:CALLS_AT]->(n18), (p0)-[:CALLS_AT]->(n15),
(p23)-[:CALLS_AT]->(n5), (p20)-[:CALLS_AT]->(n1),
(p21)-[:CALLS_AT]->(n14), (p10)-[:CALLS_AT]->(n1),
(p19)-[:NEXT]->(p10)-[:NEXT]->(p0)-[:NEXT]->(p2)-[:NEXT]->(p23),
(p22)-[:NEXT]->(p17)-[:NEXT]->(p18), (p21)-[:NEXT]->(p20);
```

```
$ sudo systemctl start neo4j
```

```
$ cypher-shell -u neo4j -p neo4j123
```

Connected to Neo4j using Bolt protocol version 6.0 at neo4j://localhost:7687 as user neo4j.

Type :help for a list of available commands or :exit to exit the shell.

Note that Cypher queries must end with a semicolon.

```
neo4j@neo4j>
```

```
neo4j@neo4j>
>....
orth Road'}),
  (n15:Station {name: 'Clapham High Street'}),
  (n16:Station {name: 'Peckham Rye'}),
  (n17:Station {name: 'Brixton'}),
  (n14:Station {name: 'London Victoria'}),
  (n18:Station {name: 'Clapham Junction'}),
  (p10:Stop {departs: time('22:37'), arrives: time('22:36')}),
  (p0:Stop {departs: time('22:41'), arrives: time('22:41')}),
  (p2:Stop {departs: time('22:43'), arrives: time('22:43')}),
  (p17:Stop {arrives: time('22:50'), departs: time('22:50')}),
  (p18:Stop {arrives: time('22:46'), departs: time('22:46')}),
  (p19:Stop {departs: time('22:33'), arrives: time('22:31')}),
  (p21:Stop {arrives: time('22:55')}),
  (p20:Stop {departs: time('22:44'), arrives: time('22:43')}),
  (p22:Stop {arrives: time('22:55')}),
  (p23:Stop {arrives: time('22:48')}),
  (n15)-[:LINK {distance: 1.96}]->(n1)-[:LINK {distance: 0.86}]->(n16),
  (n15)-[:LINK {distance: 0.39}]->(n6)-[:LINK {distance: 0.7}]->(n5)-[:LINK {distance: 1.24}]->(n14), (n5)-[:LINK {distance: 1.45}]->(n18),
  (n14)-[:LINK {distance: 3.18}]->(n17)-[:LINK {distance: 1.11}]->(n1),
  (p2)-[:CALLS_AT]->(n6), (p17)-[:CALLS_AT]->(n5), (p19)-[:CALLS_AT]->(n16),
  (p22)-[:CALLS_AT]->(n14), (p18)-[:CALLS_AT]->(n18), (p0)-[:CALLS_AT]->(n15), (p23)-[:CALLS_AT]->(n5), (p20)-[:CALLS_AT]->(n1),
  (p21)-[:CALLS_AT]->(n14), (p10)-[:CALLS_AT]->(n1), (p19)-[:NEXT]->(p10)-[:NEXT]->(p0)-[:NEXT]->(p2)-[:NEXT]->(p23),
  (p22)-[:NEXT]->(p17)-[:NEXT]->(p18), (p21)-[:NEXT]->(p20);
0 rows
ready to start consuming query after 933 ms, results consumed after another 0 ms
Created 18 nodes, created 25 relationships, set 33 properties, added 18 labels
```

Matching Fixed-Length Paths

1. The following query gets the arrival time of all trains calling at Denmark Hill:

```
MATCH (s:Stop)-[:CALLS_AT]->(:Station {name: 'Denmark Hill'})
```

```
RETURN s.arrives AS arrivalTime
```

Let us dissect the query above.

(s:Stop) Finds a node labeled Stop. In rail models, a Stop typically represents one scheduled call at a station — i.e., "this specific train stops here at this specific time."

-[:CALLS_AT]-> Follows a CALLS_AT relationship outward from the stop to the station it serves.

(:Station {name: 'Denmark Hill'}) The station node at the other end, filtered by name. Note there's no variable here — the station is only used as a filter, never returned.

RETURN s.arrives AS arrivalTime Outputs the arrives property of each stop (the scheduled arrival time), renamed to arrivalTime.

```
neo4j@neo4j> MATCH (s:Stop)-[:CALLS_AT]->(:Station {name: 'Denmark Hill'})
RETURN s.arrives AS arrivalTime;
+-----+
| arrivalTime |
+-----+
| 22:36Z      |
| 22:43Z      |
+-----+
2 rows
ready to start consuming query after 405 ms, results consumed after another 9 ms
```

2. Path patterns can include inline WHERE clauses. The following query gets the next calling point of the service that departs Denmark Hill at 22:37:

```
MATCH (n:Station {name: 'Denmark Hill'})<-[:CALLS_AT]-
  (s:Stop WHERE s.departs = time('22:37'))-[:NEXT]->(:Stop)-[:CALLS_AT]->(d:Station)
RETURN d.name AS nextCallingPoint
```

Let us dissect the query above.

(n:Station {name: 'Denmark Hill'}) Finds the physical station node for Denmark Hill. Bound to n so it can be referenced in the pattern.

<-[:CALLS_AT]- Walks backwards along CALLS_AT - from the station to the scheduled stop that calls there. The reversed arrow matches the stored direction (Stop)-[:CALLS_AT]->(Station).

(s:Stop WHERE s.departs = time('22:37')) Binds the stop to s and filters it to the one whose departs property equals 22:37. Note the WHERE is written inline inside the node pattern here - a shorthand form that's equivalent to a separate WHERE clause. The time('22:37') converts the string to a Cypher time value for comparison.

-[:NEXT]-> Follows the NEXT relationship outward from that stop to the following stop on the same trip. ***This is how the schedule is chained in order.***

(:Stop) The next scheduled call — anonymous, because we don't need its properties, only its connection onward.

-[:CALLS_AT]->(d:Station) Follows CALLS_AT from that next stop to the station it serves, bound to d.

RETURN d.name AS nextCallingPoint Outputs the name of that next station, renamed to nextCallingPoint.

Matching Shortest Paths

The Cypher keyword SHORTEST is used to find variations of the shortest paths between nodes. This includes the ability to look for the shortest, second-shortest (and so on) paths, all available shortest paths, and groups of paths containing the same pattern length. The ANY keyword can be used to test the reachability of nodes from a given node(s).

1. The following query finds the shortest route through the network from Brixton to Clapham Junction, following any number of LINK connections, and returns the list of station names along that route.

```
MATCH p = SHORTEST 1
  (:Station {name: "Brixton"})
  (()-[:LINK]-(:Station))*
  (:Station {name: "Clapham Junction"})
RETURN [station IN nodes(p) | station.name] AS route;
```

Let us understand the query above.

MATCH p = SHORTEST 1 Says: find the single shortest path, and bind it to the variable p. SHORTEST 1 means "one shortest path" — if there are ties, you get one of them. SHORTEST k would return up to k equally short paths.

(:Station {name: "Brixton"}) The starting node — the station named Brixton. Anonymous, since we only need it as an anchor.

(()-[:LINK]-(:Station))* A quantified path pattern: repeat the segment ()-[:LINK]-(Station) one or more times. The () is an anonymous intermediate node, [:LINK] is the connection type, and (:Station) is the station at each step. The + means "one or more repetitions."

(:Station {name: "Clapham Junction"}) The ending node — the station named Clapham Junction.

RETURN [station IN nodes(p) | station.name] AS route Extracts the list of nodes from the path p, then maps each node to its name property using a list comprehension. The result is a list of station names, renamed to route.

```
neo4j@neo4j> MATCH p = SHORTEST 1
    (:Station {name: "Brixton"})
    (()-[:LINK]-(:Station))+
    (:Station {name: "Clapham Junction"})
    RETURN [station IN nodes(p) | station.name] AS route
    ;
+-----+
| route                                     |
+-----+
| ["Brixton", "London Victoria", "Battersea Park", "Clapham Junction"] |
+-----+

1 row
ready to start consuming query after 385 ms, results consumed after another 22 ms
```

2. To find all shortest paths, the **ALL SHORTEST** keywords can be used:

```
MATCH p = ALL SHORTEST
    (:Station {name: "Denmark Hill"})
    (()-[:LINK]-(:Station))+
    (:Station {name: "Clapham Junction"})
RETURN [station IN nodes(p) | station.name] AS route;
```

```
neo4j@neo4j> MATCH p = ALL SHORTEST
    (:Station {name: "Denmark Hill"})
    (()-[:LINK]-(:Station))+
    (:Station {name: "Clapham Junction"})
    RETURN [station IN nodes(p) | station.name] AS route;
+-----+
| route                                     |
+-----+
| ["Denmark Hill", "Clapham High Street", "Wandsworth Road", "Battersea Park", "Clapham Junction"] |
| ["Denmark Hill", "Brixton", "London Victoria", "Battersea Park", "Clapham Junction"] |
+-----+

2 rows
ready to start consuming query after 135 ms, results consumed after another 17 ms
```

3. In general, **SHORTEST k** can be used to return the **k** shortest paths. The following returns the two shortest paths:

```
MATCH p = SHORTEST 2
    (:Station {name: "Denmark Hill"})
    (()-[:LINK]-(:Station))+
    (:Station {name: "Clapham High Street"})
RETURN [station IN nodes(p) | station.name] AS route;
```

```
neo4j@neo4j> MATCH p = SHORTEST 2
    (:Station {name: "Denmark Hill"})
    (()-[:LINK]-(:Station))+
    (:Station {name: "Clapham High Street"})
    RETURN [station IN nodes(p) | station.name] AS route;
+-----+
| route |
+-----+
| ["Denmark Hill", "Clapham High Street"] |
| ["Denmark Hill", "Brixton", "London Victoria", "Battersea Park", "Wandsworth Road", "Clapham High Street"] |
+-----+

2 rows
ready to start consuming query after 165 ms, results consumed after another 33 ms
```

Queries to Solve

1. Write a query that finds the shortest route between **Brixton** and **Clapham Junction** by **number of hops**.
2. Write a query that finds the shortest route between **Brixton** and **Clapham Junction** by **total distance**.